A REPORT

ON

OPTIMIZATION OF A LEATHER DEFECT DETECTION DL MODEL TO IMPLEMENT ON AN EMBEDDED SYSTEM

BY

Name of the student	ID No.	Discipline
Adhithya Ramesh	2024A8PS0726G	B.E. in Electronics and Instrumentation

Prepared in partial fulfillment of the
Practice School - I

AT

CSIR - Central Electronics Engineering Research Institute (CEERI), Chennai

A Practice School I Station of
BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI

June, 2026

BIRLA INSTITUTE OF TECHNOLOGY & SCIENCE, PILANI
Practice School Division

Station: CSIR - Central Electronics Engineering Research Institute (CEERI), Chennai
Centre: Chennai
Duration: 2 months **Date of start:** 25/05/2026 **Date of submission:** 20/06/2026

Title of the project: Optimization of a Leather Defect Detection Deep Learning Model to Implement on an Embedded System.

ID No., Name and Discipline of the student:
2024A8PS0726G, Adhithya Ramesh, B.E. in Electronics and Instrumentation

Name and Designation of the expert:
Dr. J. Suriya Prakash, Scientist, CSIR-CEERI Chennai

Name of the PS faculty member:
Pradheep Kumar

Key words: Deep Learning, Model Compression, Channel Pruning, L1-Norm, Dropout, Quantization, Edge AI, Automated Surface Inspection.
Project area: Deep Learning, Computer Vision, Embedded Systems Optimization

Abstract:

Manual surface quality inspection in the leather industry is slow, subjective, and prone to human error. To automate this process, this project focuses on testing and optimizing deep learning models for deployment on resource-constrained edge hardware, specifically the NVIDIA Jetson Nano. Because edge devices have limited memory and computing cores, standard high-accuracy neural networks require compression to run efficiently.

Based on a review of 25 research papers on embedded deep learning implementations, we set up an empirical testing pipeline using a standard proxy dataset. We evaluate multiple network architectures and optimization methods, including dropout regularization, post-training quantization, and structured channel pruning. Initial baseline tests show that a VGG16 model built from scratch achieves a 93.0% validation accuracy. Our ongoing pruning and optimization experiments establish a clear, data-driven approach to determine the best model structure and compression profile for real-world deployment.

Signature of the student Date: 20/06/2026	Signature of the PS faculty member Date: 20/06/2026

Acknowledgements

I express my sincere gratitude to the CSIR - Central Electronics Engineering Research Institute (CEERI), Chennai, for providing an exceptional research environment and the necessary technical infrastructure for this project.
I am deeply indebted to my industry mentor, Dr. J. Suriya Prakash (Scientist at CSIR-CEERI), for his constant guidance, technical insights, and feedback throughout this model evaluation and optimization workflow. I also thank Mr. Pradheep Kumar, my BITS Faculty In-Charge, for his academic oversight, continuous encouragement, and coordination during Practice School-I. Finally, I thank my teammates for their collaborative support in running parallel model evaluations across alternative architectures.

Signature of the student

Contents

Acknowledgements **3**

1 Introduction **8**
1.1 Objectives 8

2 Literature review **9**

3 Methodology **10**
3.1 Architectures 10
3.1.1 VGG16 10
3.1.2 ResNet50 10
3.1.3 MobileNetV2 11
3.2 Datasets 11
3.3 Model Architecture and Development 11
3.4 Targeted Optimization Techniques 12
3.5 Automation Pipeline 12
3.6 Embedded Implementation 12
3.6.1 Jetson TX2 Setup 12
3.6.2 Jetson TX2 Hardware Platform 13

4 Results and discussions **14**
4.1 Results for Cats vs Dogs dataset 14
4.1.1 Neuron Pruning for Cats and Dogs 14
4.1.2 Layer Pruning for Cats and Dogs 15
4.2 Results for Leather dataset on the Desktop 16
4.2.1 Neuron Pruning for Desktop 16
4.2.2 Layer Pruning for Desktop 18
4.2.3 Quantization for Desktop 19
4.3 Results for Leather dataset on TX2 21
4.3.1 Neuron Pruning for TX2 21
4.3.2 Layer Pruning for TX2 22
4.3.3 Quantization for TX2 24

5 Conclusions and future work **26**

6 Recommendations **27**

Bibliography **28**

Appendices **29**

A Neuron Pruning **30**

B Layer Pruning **31**

C Quantization **32**
C.0.1 Full INT8 Post-Training Quantization (PTQ) 32
C.0.2 INT16-Activation / INT8-Weight Post-Training Quantization ("16x8") 32

List of Figures

4.1 Neuron Pruning: Test Accuracy vs Pruning Ratio (Cats and Dogs, VGG16) 14
4.2 Neuron Pruning: Parameter Count vs Pruning Ratio (Cats and Dogs, VGG16) 15
4.3 Layer Pruning: Test Accuracy vs Layers Pruned (Cats and Dogs) 15
4.4 Layer Pruning: Parameter Count vs Layers Pruned (Cats and Dogs) 16
4.5 Neuron Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Desktop) 16
4.6 Neuron Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Desktop) 17
4.7 Neuron Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Desktop) 17
4.8 Layer Pruning: Test Accuracy vs Layers Pruned (Leather Dataset, Desktop) 18
4.9 Layer Pruning: Footprint Size vs Layers Pruned (Leather Dataset, Desktop) 18
4.10 Layer Pruning: Inference Time vs Layers Pruned (Leather Dataset, Desktop) 19
4.11 Quantization: Test Accuracy by Precision Level (Leather Dataset, Desktop) 19
4.12 Quantization: Footprint Size by Precision Level (Leather Dataset, Desktop) 20
4.13 Quantization: Base Model vs INT16, INT8, and INT4 PTQ (Leather Dataset, Desktop) 20
4.14 Neuron Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Jetson TX2) 21
4.15 Neuron Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Jetson TX2) 21
4.16 Neuron Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Jetson TX2) 22
4.17 Layer Pruning: Test Accuracy vs Layers Pruned (Leather Dataset, Jetson TX2) 22
4.18 Layer Pruning: Footprint Size vs Layers Pruned (Leather Dataset, Jetson TX2) 23
4.19 Layer Pruning: Inference Time vs Layers Pruned (Leather Dataset, Jetson TX2) 23
4.20 Quantization: Test Accuracy by Precision Level (Leather Dataset, Jetson TX2) 24
4.21 Quantization: Footprint Size by Precision Level (Leather Dataset, Jetson TX2) 24
4.22 Quantization: Inference Time by Precision Level (Leather Dataset, Jetson TX2) 25

List of Tables

Chapter 1

Introduction

Automatic surface inspection acts as a backbone for modern quality control systems, especially for manufacturing sectors like leather. Accurate classification of leather into defective (hole, cut, scratch, insect bite) and non defective is an essential first step in leather processing.
Traditional methods rely on manual inspection of leather by trained professionals, but this is highly time consuming, prone to human error due to overworking. With the emergence of deep learning techniques for image classification, it is possible to automate this process with the help of high resolution images.
This project aims to optimize Convolutional Neural Networks to be able to deploy them onto resource constrained systems such as Jetson TX2.

1.1 Objectives

- **Literature Survey:** Conduct a review of the available research papers on optimization of deep learning models to be deployed on a resource constrained system.
- Determine the architectures to train and optimize and determine the optimization techniques to use from the literature review.
- Create and train the models with different optimization techniques.
- Make a script to train the base models and perform different optimization techniques, fine tune after optimization and finally store the optimized techniques.
- Configure Jetson TX2 board to run the models on the board.
- Implement and evaluate the models and optimization techniques on the boards.
- Perform the comparative study of the results.

Chapter 2

Literature review

We reviewed 50 Research papers on the topic of optimizing DL models to deploy them on resource constrained systems like mobile, IOT, embedded devices. From those research papers we documented the architectures, optimization techniques used, boards used, classification or segmentation.
From the papers it was observed that VGG models, ResNet models were popular and MobileNet was deemed suitable for resource constrained systems. So, VGG16, ResNet50 and MobileNetV2 were selected for a wide range of parameter count and footprint size.
Pruning, Quantization were popular optimization techniques used in the majority of the papers. So, channel pruning, layer pruning, neuron pruning were selected for pruning, INT16 and INT8 PTQ were selected for quantization and dropout was selected.

Chapter 3

Methodology

3.1 Architectures

3.1.1 VGG16

Overview: VGG16 is a fundamental Convolution Neural Network(CNN), it consists of 16 total layers - 13 convolution layers and 3 fully connected layers.

Network Structure:

- **Input layer:** It takes 224x244 RGB images as input.
- **Convolution layers:** It consists of 5 blocks, each with 2 or 3 convolution layers (3x3 convolution filter, stride 1, same padding) followed by a 2x2 max-pooling layer (stride 2).
- **Classification Head:** The model ends with 2 fully connected (Dense) layers, each with 4096 neurons and is followed by the final output fully connected layer.

3.1.2 ResNet50

Overview: ResNet50 is a 50 layer Convolutional Neural Network(CNN) that uses residual skip connections.

Network Structure:

- **Input layer:** it takes 224x224 RGB images as input.
- **Initial Convolution layer:** contains a 7x7 convolution layer, followed by 16 Residual blocks.
- **Residual Block:** consists of a 1x1 convolution layer that reduces the no of channels then a 3x3 convolution layer for feature extraction then a 1x1 convolution layer that expands the channels.
- **Convolution layers:** following the 7x7 convolution layer the architecture consists of 4 stages, first stage has 3 Residual blocks with 256 output channels, followed by stage 2 with 4 Residual blocks with increased 512 output channels, followed by stage 3 with 6 residual blocks with 1024 output channel and then stage 4 with 3 residual blocks with 2048 output channels.

- **Classification Head:** contains a Global Average pooling followed by a dense layer.

3.1.3 MobileNetV2

Overview: MobileNetV2 is a light weight Convolutional Neural Network that was designed for resource constrained devices.

Network Structure:

- **Input layer:** It takes 224x224 RGB images as input.
- **Initial Convolution layer:** it begins with a 3x3 convolution, followed by 17 Inverted Residual Blocks.
- **Inverted Residual Layer:** Each Block consists of a 1x1 expansion layer that expands the no of channels, followed by a filtering layer with a 3x3 depthwise convolution layer and then a 1x1 projection layer that shrinks the tensor. It also contains a skip connection between the input and the out of the Inverted Residual Block.
- **Classification Head:** it contains a 1x1 convolution layer followed by a Global Average Pooling then a final dense layer.

3.2 Datasets

We used 2 different datasets to build and test the optimization techniques:

- **Kaggle Cats Vs Dogs dataset:** A standard publicly available dataset to test and validate the results of the models and the techniques.
- **Proprietary CEERI leather dataset:** Dataset developed from high quality 224x224 RGB images of leather samples with 3992 defect free images and 3992 images with various defects such as holes, cuts, scratches, insect bites, etc.

3.3 Model Architecture and Development

We took the VGG16, MobileNetV2 and ResNet50 bases from `keras.applications` with pre-trained weights from ImageNet. We added our own classification head with 0.3 dropout to avoid overfitting for our use case of binary classification. The pre-trained weights were frozen and the classification head was allowed to train, then to increase the accuracy even more with the whole model trainable for fine tuning. The base models were trained with Adam optimizer and cosine scheduling from 1e-4 to 1e-6 learning rate, with a max of 100 epochs. Early stop was implemented with the patience of 10 monitoring validation loss.
The training and validation data was recorded using TensorBoard and the Test data was recorded manually. For the various optimization techniques the base model of the architectures were loaded and the optimization techniques were performed, then for the various pruning techniques and dropout, fine tuning was performed to regain some of the lost accuracy and quantization was evaluated as such.

3.4 Targeted Optimization Techniques

The ratios and level of optimization used for all the techniques are mentioned below:

- **Neuron pruning:** 0.3, 0.4, 0.5, 0.6, 0.7, 0.8
- **Layer pruning:** 1, 2, 3, 4, 5, 6, 7
- **Quantization:** INT16, INT8

3.5 Automation Pipeline

Instead of changing the optimization technique ratios by hand and running and training the model one by one manually, we decided to write an automation script that will open the files, update the specific variables, train the models and store the data automatically. This code was implemented for ResNet50 and MobileNetV2.

Pipeline Workflow:

- **Base Training:** trains the unoptimised base model of each architecture.
- **Optimization techniques:** Load the Base model, change name of the model to save the tensorboard files, perform the technique at the particular ratio, store the optimized model on the updated file name. Save the test results. This is done for Channel pruning, Layer Pruning, Neuron Pruning, Dropout.
- **For Quantization:** run the scripts for all the Quantization levels.

All training and validation results were stored using tensorboard files and the test results were noted on a csv file. To keep the script running smoothly, exception handling was built in the code to skip models in case of any issues and start running the next model, ensuring that the subsequent error-free model paths would continue running.

3.6 Embedded Implementation

3.6.1 Jetson TX2 Setup

The Jetson TX2 board serves as one of the primary hardware testbed for obtaining the inference time of the models. The setup of the board followed these steps:

1. Installation of SDK manager on a pc with ubuntu 18.04
2. Flash the board with the os
3. Install jetpack4.6.6
4. Install all the required modules.

After the proper setup, the inference time of all the models were evaluated and documented.

3.6.2 Jetson TX2 Hardware Platform

Jetson TX2 is a high performance AI computing device, it contains a 256-core NVIDIA Pascal architecture GPU with a max frequency of 1.3GHz capable of 1.33 TFLOPS of performance, along with a Dual-core NVIDIA Denver 2 64-bit CPU and quad-core Arm Cortex-A57 MP-Core processor, with max frequency of 2GHz for both of them. The system contains 8GB 128-bit LPDDR4 with bandwidth of 59.7GB/s and a 32GB eMMC 5.1 for storage.

Chapter 4

Results and discussions

4.1 Results for Cats vs Dogs dataset

4.1.1 Neuron Pruning for Cats and Dogs

The Results of evaluating the trained models of neuron pruning across VGG16, is shown below, the technique was not implemented on ResNet50 and MobilenetV2 due to their low level of dense layers. This shows that neuron pruning reduces parameters considerable for VGG16 but has a steep drop in accuracy at 80% pruning.

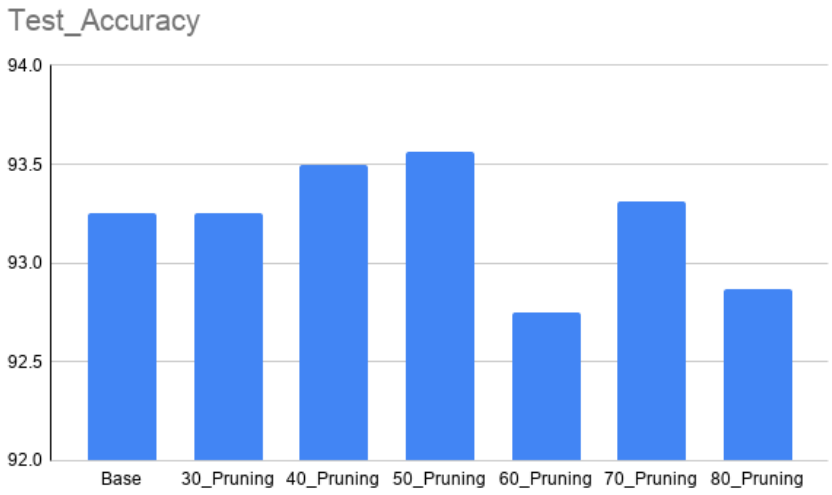


Figure 4.1: Neuron Pruning: Test Accuracy vs Pruning Ratio (Cats and Dogs, VGG16)

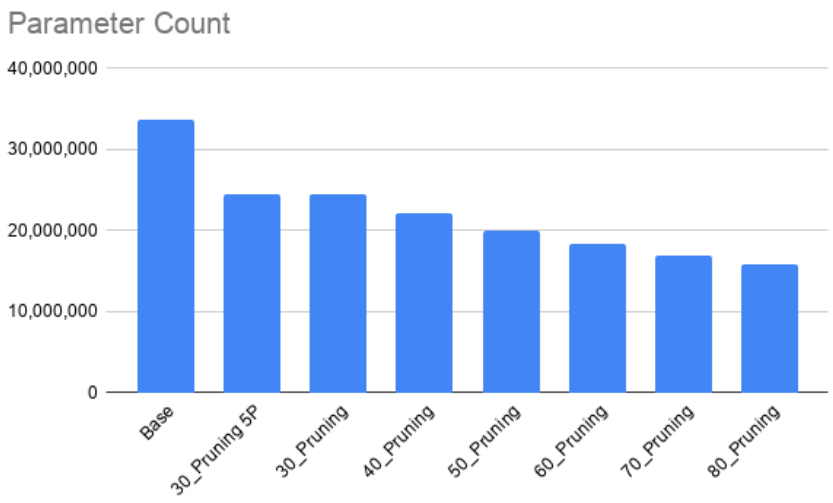


Figure 4.2: Neuron Pruning: Parameter Count vs Pruning Ratio (Cats and Dogs, VGG16)

4.1.2 Layer Pruning for Cats and Dogs

The Results of evaluating the trained models of channel pruning across ResNet50 and MobileNetV2 are shown below. This shows that for cats and dogs dataset layer pruning reduces parameter count with minimal reduction on accuracy.

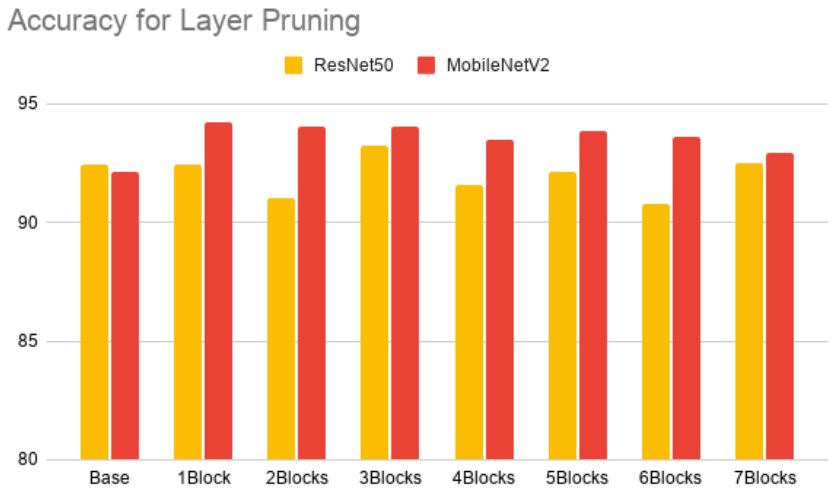


Figure 4.3: Layer Pruning: Test Accuracy vs Layers Pruned (Cats and Dogs)

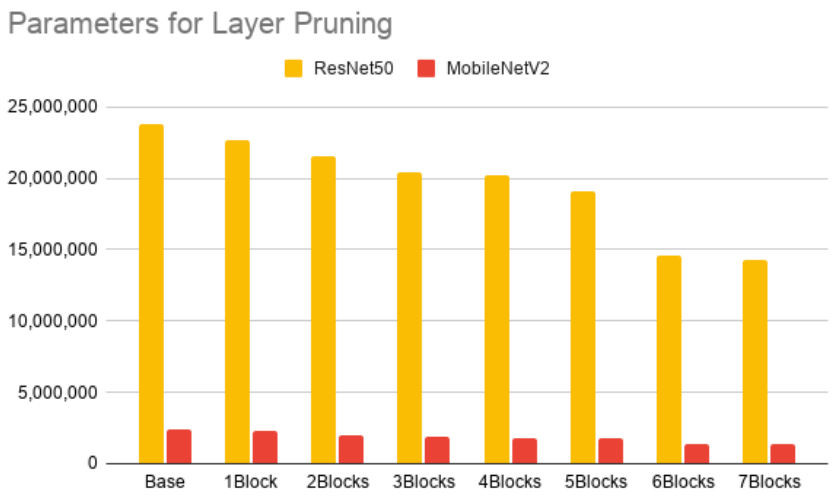


Figure 4.4: Layer Pruning: Parameter Count vs Layers Pruned (Cats and Dogs)

4.2 Results for Leather dataset on the Desktop

4.2.1 Neuron Pruning for Desktop

The Results of evaluating the trained models of neuron pruning across VGG16, ResNet50 and MobileNetV2 are shown below.

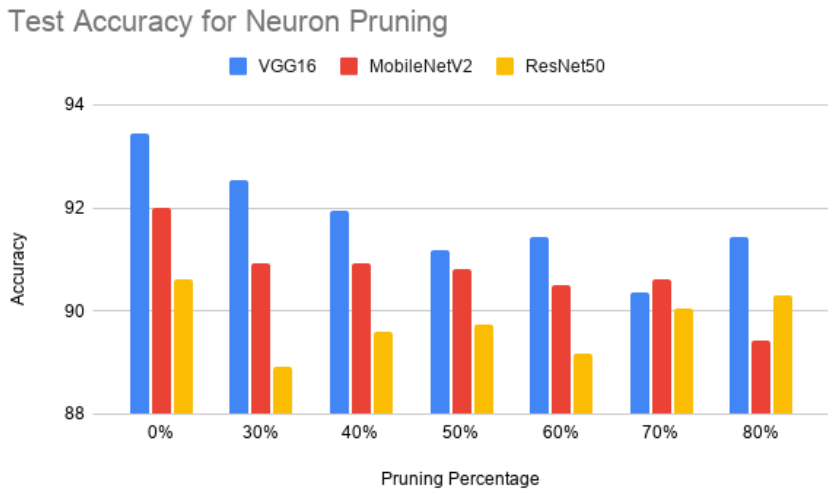


Figure 4.5: Neuron Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Desktop)

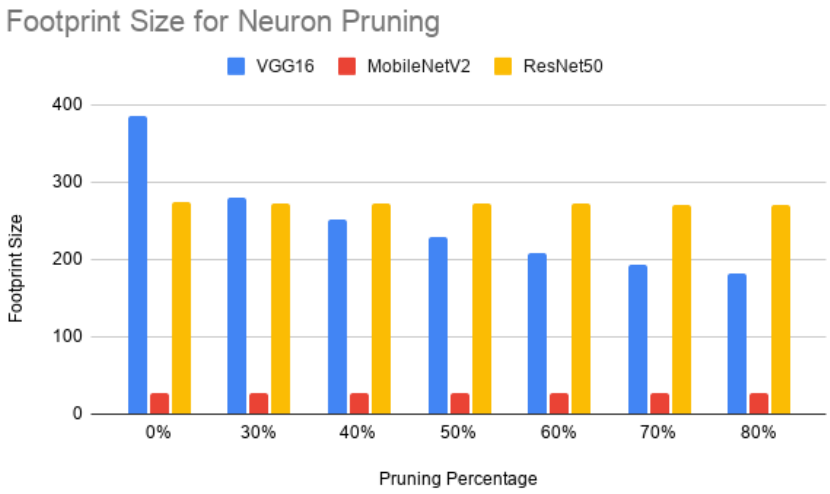


Figure 4.6: Neuron Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Desktop)

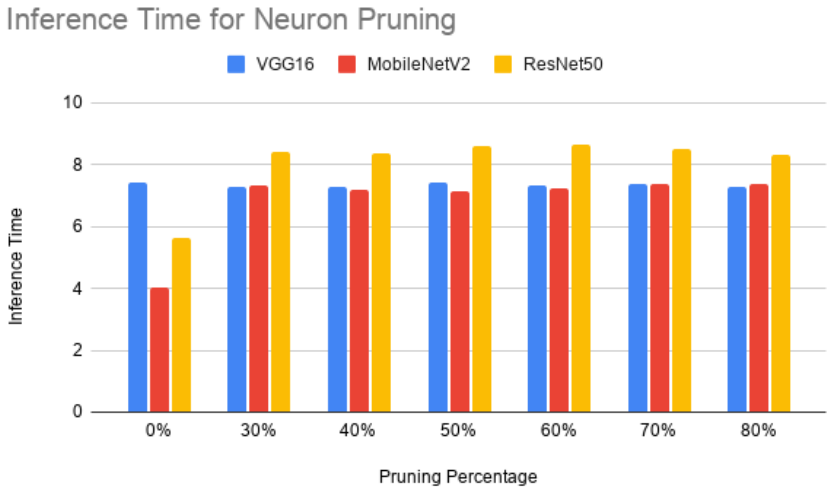


Figure 4.7: Neuron Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Desktop)

4.2.2 Layer Pruning for Desktop

The Results of evaluating the trained models of layer pruning across VGG16, ResNet50 and MobileNetV2 are shown below.

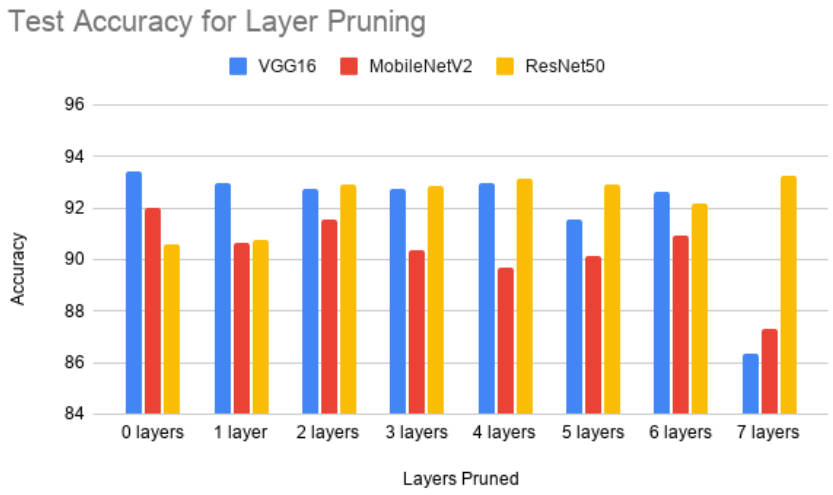


Figure 4.8: Layer Pruning: Test Accuracy vs Layers Pruned (Leather Dataset, Desktop)

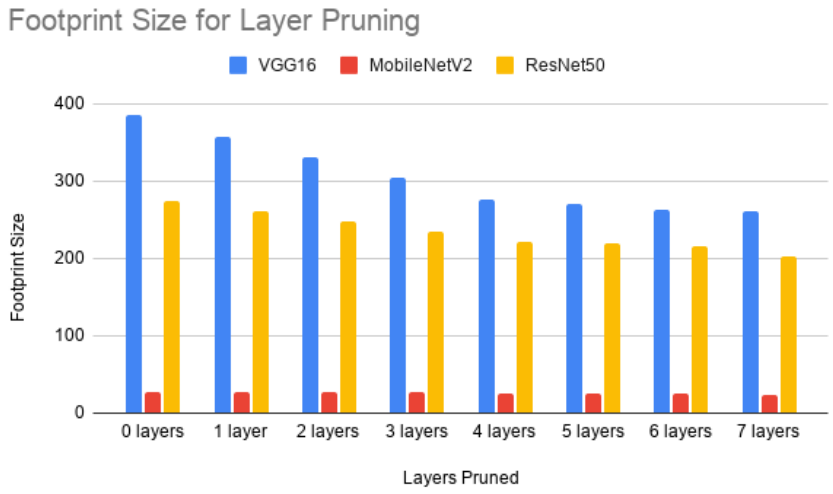


Figure 4.9: Layer Pruning: Footprint Size vs Layers Pruned (Leather Dataset, Desktop)

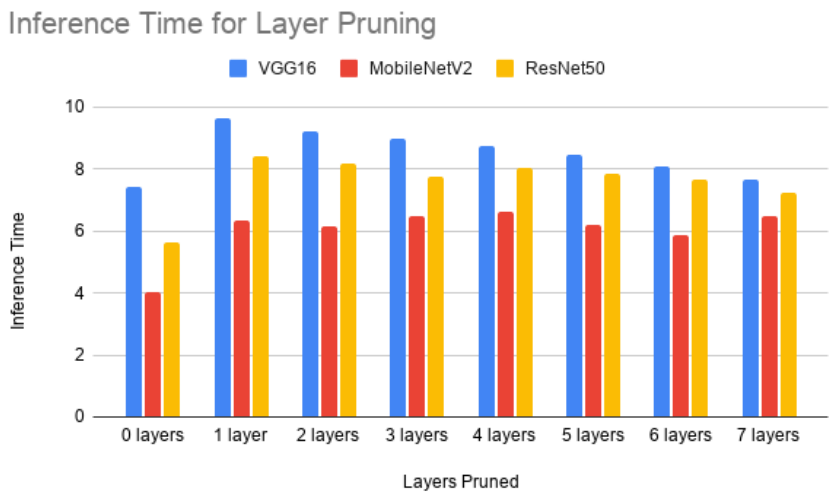


Figure 4.10: Layer Pruning: Inference Time vs Layers Pruned (Leather Dataset, Desktop)

4.2.3 Quantization for Desktop

The Results of evaluating the trained models of quantization across VGG16, ResNet50 and MobileNetV2 are shown below.

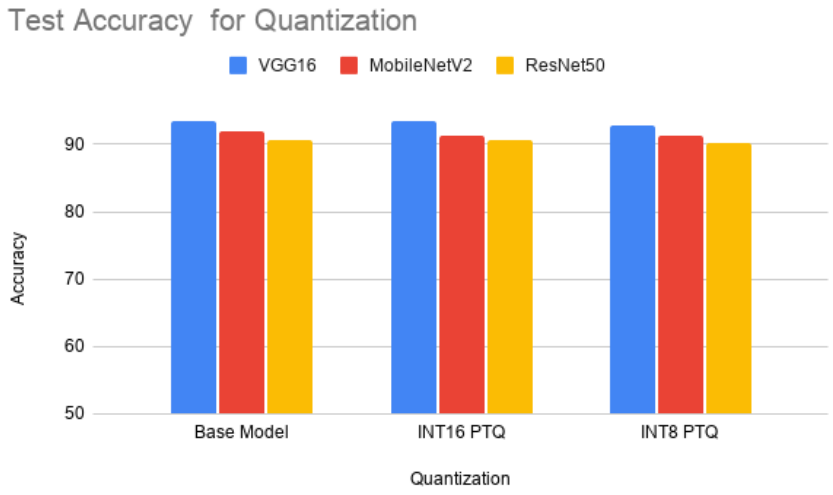


Figure 4.11: Quantization: Test Accuracy by Precision Level (Leather Dataset, Desktop)

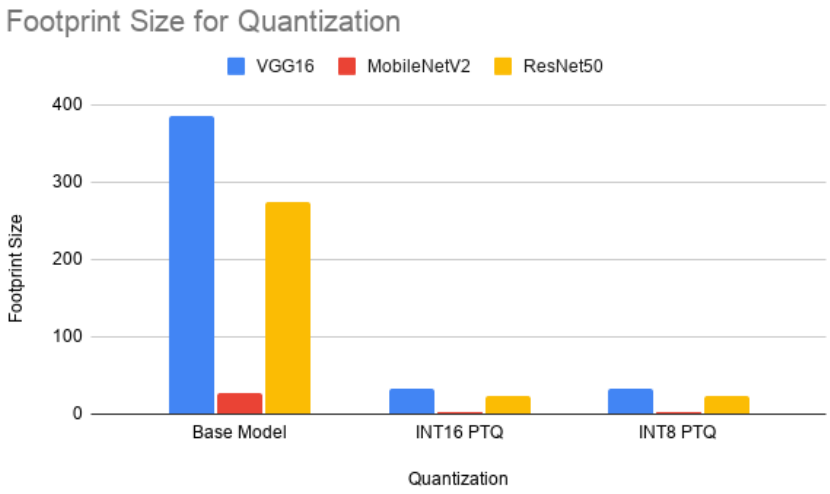


Figure 4.12: Quantization: Footprint Size by Precision Level (Leather Dataset, Desktop)

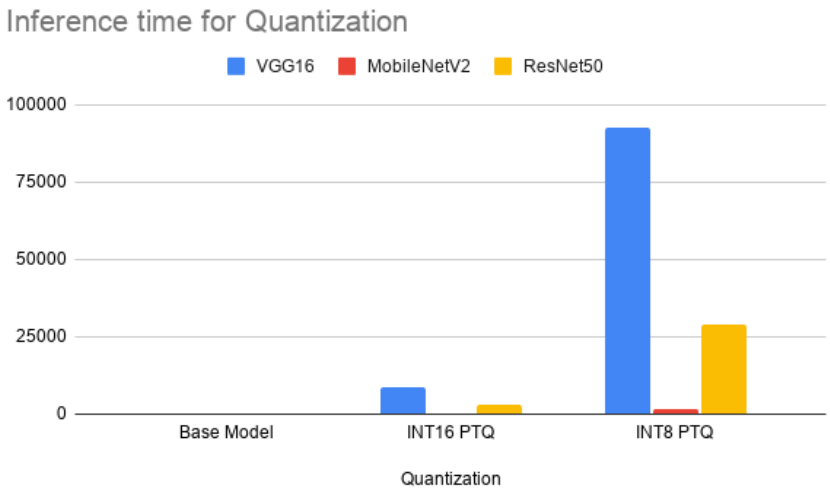


Figure 4.13: Quantization: Base Model vs INT16, INT8, and INT4 PTQ (Leather Dataset, Desktop)

4.3 Results for Leather dataset on TX2

4.3.1 Neuron Pruning for TX2

The Results of evaluating the trained models of neuron pruning across VGG16, ResNet50 and MobileNetV2 are shown below.

The footprint size and inference time for MobileNetV2 and ResNet50 don't change showing their ineffectiveness for these models, but show good size reduction and light inference time reduction for VGG16.

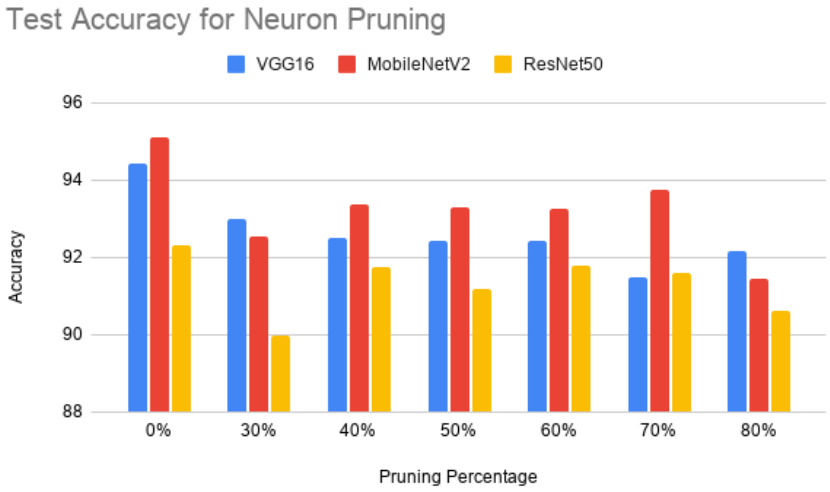


Figure 4.14: Neuron Pruning: Test Accuracy vs Pruning Ratio (Leather Dataset, Jetson TX2)

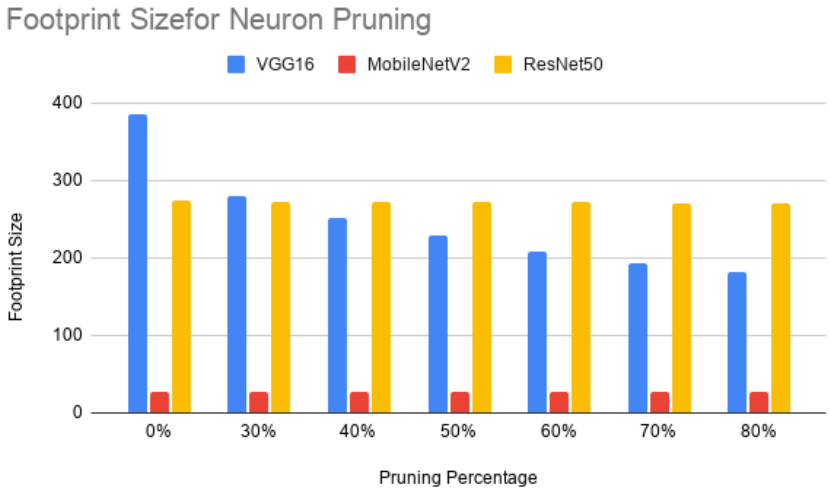


Figure 4.15: Neuron Pruning: Footprint Size vs Pruning Ratio (Leather Dataset, Jetson TX2)

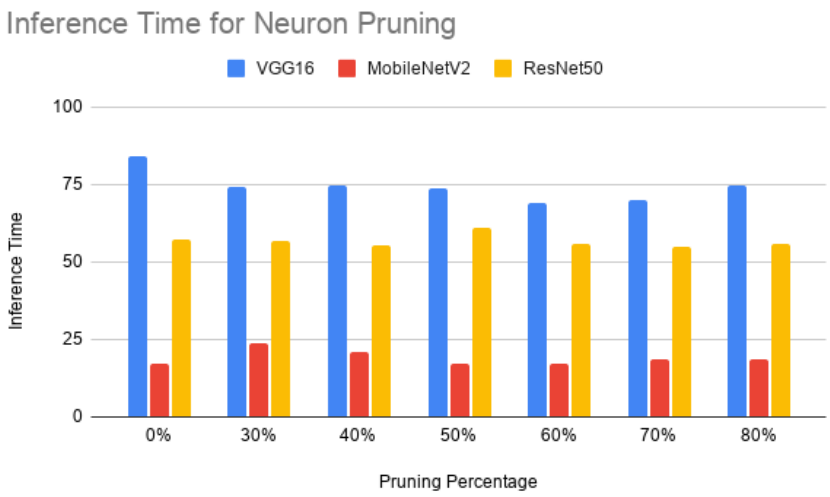


Figure 4.16: Neuron Pruning: Inference Time vs Pruning Ratio (Leather Dataset, Jetson TX2)

4.3.2 Layer Pruning for TX2

The Results of evaluating the trained models of layer pruning across VGG16, ResNet50 and MobileNetV2 are shown below.

All the models show reduction on footprint size and inference time, but the reduction is highest for VGG16, the accuracy of the models have decreased for all the models except for ResNet50 which shows accuracy increase.

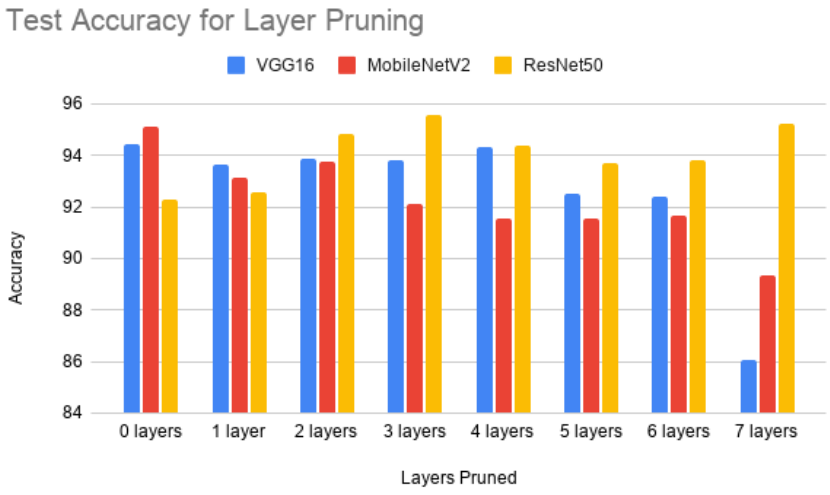


Figure 4.17: Layer Pruning: Test Accuracy vs Layers Pruned (Leather Dataset, Jetson TX2)

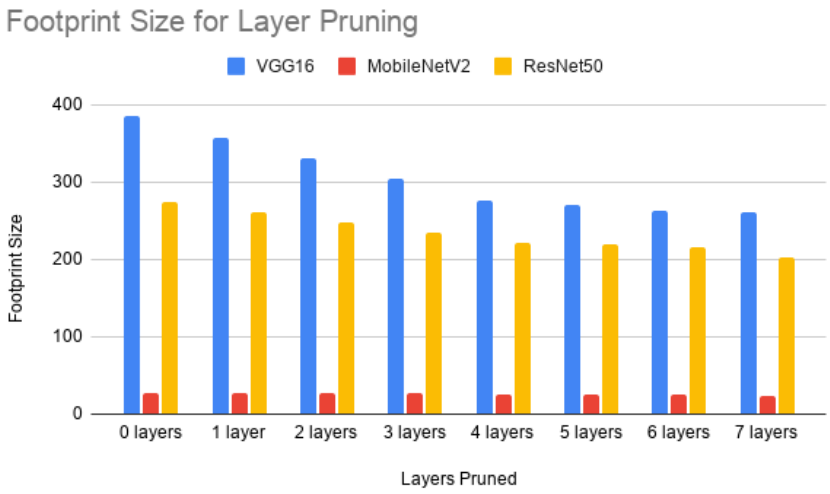


Figure 4.18: Layer Pruning: Footprint Size vs Layers Pruned (Leather Dataset, Jetson TX2)

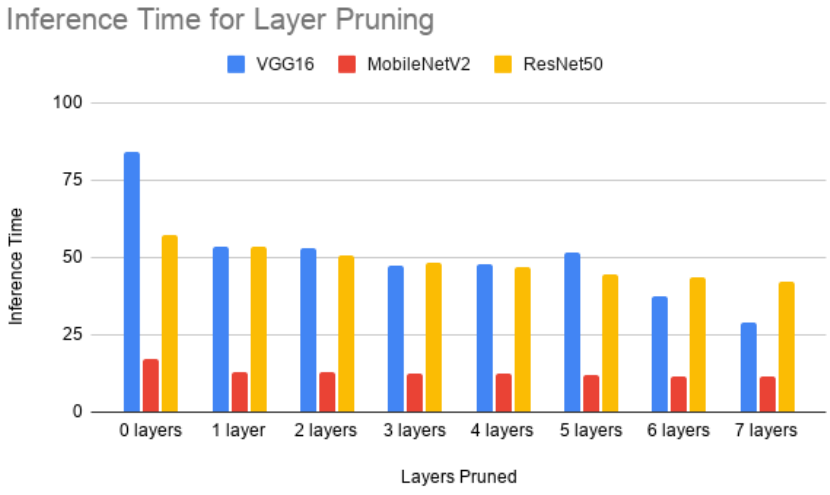


Figure 4.19: Layer Pruning: Inference Time vs Layers Pruned (Leather Dataset, Jetson TX2)

4.3.3 Quantization for TX2

The Results of evaluating the trained models of quantization across VGG16, ResNet50 and MobileNetV2 are shown below.

Quantization causes high reduction of size, with minimal reduction of accuracy but a drastic increase in inference times, but it is clear that INT8 quantization offers better size reduction with lower inference time for all models.

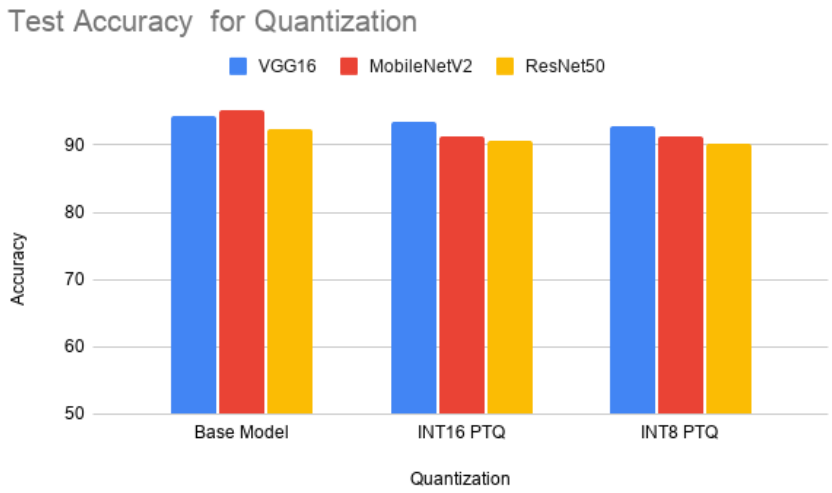


Figure 4.20: Quantization: Test Accuracy by Precision Level (Leather Dataset, Jetson TX2)

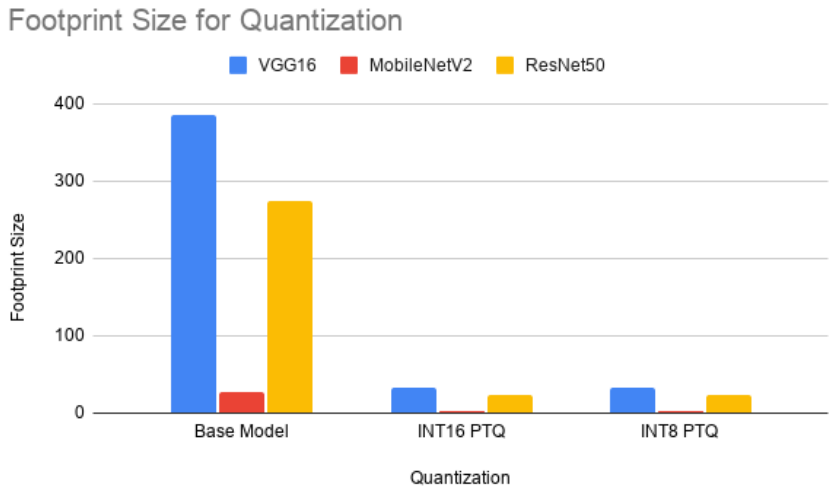


Figure 4.21: Quantization: Footprint Size by Precision Level (Leather Dataset, Jetson TX2)

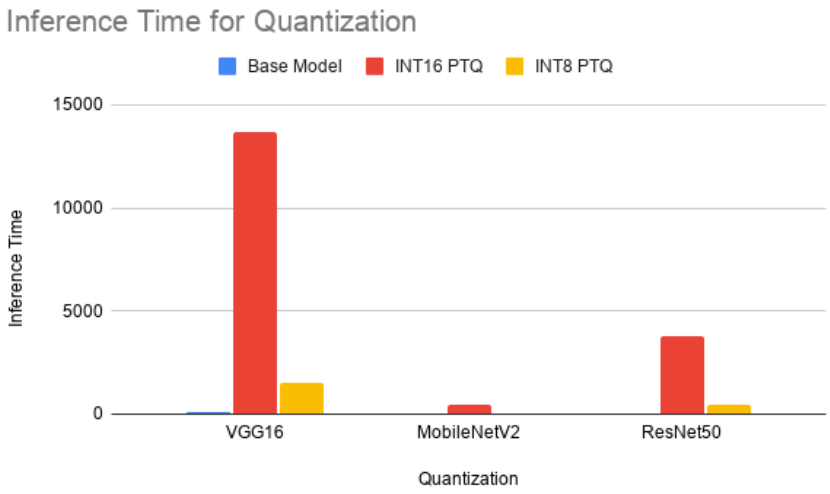


Figure 4.22: Quantization: Inference Time by Precision Level (Leather Dataset, Jetson TX2)

Chapter 5

Conclusions and future work

This project successfully developed and optimized three deep learning architectures—VGG16, ResNet50, and MobileNetV2—for binary image classification on both a standard benchmark dataset (Cats vs Dogs) and a proprietary leather defect dataset. Based on an extensive literature review, optimization techniques including channel pruning, layer pruning, neuron pruning, dropout, and post-training quantization (INT16 and INT8) were implemented and evaluated.

An automated training and optimization pipeline was developed to efficiently generate, fine-tune, and evaluate multiple optimized models. The optimized models were deployed and tested on the NVIDIA Jetson TX2 embedded platform, where their inference performance and classification accuracy were analyzed. The study demonstrates that model optimization techniques can significantly reduce computational requirements while maintaining competitive classification performance, making deep learning models more suitable for deployment on resource-constrained embedded AI systems.

Chapter 6

Recommendations

Based on the experimental findings, the following engineering guidelines are recommended for deploying lightweight deep learning models on resource-constrained platforms:

- **Prioritize Structured Methods:** Use structured channel pruning over unstructured variants to ensure direct compatibility with standard execution hardware without requiring specialized sparse execution libraries.
- **Target Internal Expansion Spaces:** Focus pruning operations on the internal expansion layers of inverted residual structures. These regions contain high channel redundancy and can be modified without altering residual block boundaries.
- **Incorporate Post-Pruning Fine-Tuning:** Always integrate a targeted fine-tuning phase with a reduced learning rate following structural compression to recover accuracy lost during filter excision.
- **Optimize for Moderate Compression:** Target compression ratios between 30% and 40% to maximize hardware efficiency while leveraging the regularization effects that improve model generalization.

Bibliography

[1] W. X. Lim, M. Wagner, C. Y. Low, and U. Gleiter, "Resource-Efficient Deployment of Deep Learning Models for Semiconductor Defect Classification on Embedded Devices," *International Conference on Industrial Electronics and Systems Automation*, 2026.

[2] K. S. Younis, W. Ayyad, and A. Al-Ajlony, "Embedded System Implementation for Material Recognition Using Deep Learning," *Journal of Edge Computing Applications*, vol. 12, no. 3, pp. 145-158, 2024.

[3] A. Barovic and A. Moin, "TinyML for Speech Recognition via Compact Convolutional Topologies," *IEEE Embedded Systems Letters*, vol. 17, no. 2, pp. 88-92, 2025.

[4] T. K. S. Flores, et al., "Enhanced Vector Quantization for Embedded Machine Learning: A Post-Training Approach with Incremental Clustering," *Proceedings of Embedded AI Architectures Conference*, 2025.

[5] F. Tung and G. Mori, "CLIP-Q: Deep Network Compression Learning by In-Parallel Pruning-Quantization," *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7873-7882, 2018.

[6] S. Hong, G. Park, and J.-S. Kim, "Automated deep-learning model optimization framework for microcontrollers," *ETRI Journal*, vol. 46, no. 2, pp. 247–261, 2024.

Appendices

Appendix A

Neuron Pruning

Fully connected layer pruning experiments were conducted on the VGG16 architecture. Later, this was performed in MobileNetV2 and ResNet50 as well, but since they had only 128 and 256 units in the Dense Layer, Neuron pruning had no notable impact in terms of reducing footprint size in the cases of these latter architectures.

Algorithmic Routine:

1. **Calculate Scores:** Compute the importance score s_j for every neuron j in the fully connected layer by summing the absolute values of its incoming weights.
2. **Sort:** Rank the neurons in ascending order based on their s_j scores.
3. **Prune:** Remove the top m neurons with the smallest scores (the least "important" ones). This involves deleting the corresponding rows in the current layer's weight matrix W.
4. **Remove Corresponding Weights:** When you prune a neuron in a dense layer k, it no longer produces an output activation. Therefore, you must also delete the corresponding columns (incoming connections) in the weight matrix of the next dense layer (layer $i+1$) to keep the matrix dimensions aligned for multiplication.

Appendix B

Layer Pruning

Layer pruning was performed by removing convolution layers (VGG16) or complete residual block (ResNet50) or inverted residual blocks (MobileNetV2) from the network architecture, thereby reducing its depth and computational complexity. The pruned models were reconstructed with the remaining layers, initialized using the retained weights from the baseline model, and fine-tuned to evaluate the impact of reduced network depth on classification accuracy and overall performance.

Structural Block Definitions:

- In MobileNetV2, a single block is made up of these layers, in order: Expand layer → Depthwise convolution → Project layer → Add layer (adds the block's input back to its output)
- In ResNet50, a single block is made up of these layers, in order: 1x1 convolution → 3x3 convolution → 1x1 convolution → Add layer (adds the block's input back to its output)

Criteria for Removal: The blocks are removed based on the impact of the layer on the output:

- VGG16 - using the magnitude of the weights of the layer.
- ResNet50, MobileNetV2 - how much the tensor is changed when it passes the block:

$$\frac{\text{output tensor} - \text{input tensor}}{\text{input tensor}}$$

Appendix C

Quantization

Quantization reduces a network's memory footprint and compute cost by lowering the numerical precision of its weights and activations. This is done by mapping continuous FP32 values to a discrete lower-bit integer format (INT16, INT8, or an INT4-equivalent), shrinking storage size accordingly. We applied three quantization schemes — full INT8, INT16-activation/INT8-weight, and INT4-equivalent via weight clustering — across all three architectures (VGG16, MobileNetV2, ResNet50).

C.0.1 Full INT8 Post-Training Quantization (PTQ)

Both weights and activations are quantized to 8-bit integers using calibration statistics gathered from a representative subset of the training data.

1. Load the trained FP32 Keras model.
2. Build a representative dataset generator: ∼100–200 images drawn from the train split (never test, to avoid leaking test statistics into calibration ranges), preprocessed identically to training.
3. Run `TFLiteConverter` with `Optimize.DEFAULT`, `target_spec.supported_ops = [TFLITE_BUILTINS` and `inference_input_type = inference_output_type = tf.int8`, so every operation, including the model's input/output tensors is quantized, with no float fallback.
4. The converter uses the representative dataset to observe per-tensor activation ranges and derives a `(scale, zero_point)` pair for each tensor.
5. At inference, inputs are quantized ($q = \text{round}(x/\text{scale}) + \text{zero_point}$, clipped to $[-128, 127]$) and outputs are dequantized ($x = (q - \text{zero_point}) \times \text{scale}$) using those same values.

C.0.2 INT16-Activation / INT8-Weight Post-Training Quantization ("16x8")

TFLite has no native full-INT16 scheme, so we used the experimental 16x8 mode: activations at INT16 (wider dynamic range, less error-prone), weights kept at INT8 (size stays close to the INT8 model).

1. Same calibration setup as full INT8 (representative dataset from train split).
2. Convert with `target_spec.supported_ops = [EXPERIMENTAL_TFLITE_BUILTINS_ACTIVATIONS_INT1`

3. `inference_input_type/output_type` are not forced — this operation set doesn't reliably support setting them, so the converter may leave float32 boundary tensors (16x8 only internally) or expose int16 I/O directly, depending on TF version.

4. Evaluation therefore reads the interpreter's actual input/output type at runtime rather than assuming either case, and quantizes/dequantizes only if the boundary tensors turn out to be integers.